TypeScript Hands-On Exercises - Answers

1. Data Types & Variable Declaration

```
let name: string = "Bala";
let age: number = 30;
let isTester: boolean = true;
let salary: number = 50000;
let city: string = "Chennai";

console.log(`Employee ${name} is ${age} years old`);
```

2. Primitive Types, Type Annotations & Type Inference

```
let course: string = "TypeScript";
let duration: number = 30;
let isCompleted: boolean = false;

let company = "OpenAI"; // inferred as string

// Fix Type Error
let employeeAge: number = 30;
```

3. String, Math, Date Functions

```
let message = "typescript";

console.log(message.toUpperCase());
console.log(message.toLowerCase());
console.log(message.substring(0, 4));
console.log(message.replace("type", "java"));
console.log(message.includes("script"));

// Random Number
console.log(Math.floor(Math.random() * 100) + 1);

// Date
const today = new Date();
console.log(today.getFullYear());
console.log(today.getMonth() + 1);
```

4. If Conditions

```
let age = 20;

if (age >= 18) {
  console.log("Eligible to vote");
}

let num = 10;

if (num % 2 === 0) {
  console.log("Even");
} else {
  console.log("Odd");
}
```

5. Switch Case

```
let day = 3;

switch(day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Invalid");
}
```

6. Loops

```
for(let i = 1; i <= 10; i++) {
  console.log(i);
}

// Multiplication Table
for(let i = 1; i <= 10; i++) {
  console.log(`5 x ${i} = ${5 * i}`);
}

// Star Pattern
for(let i = 1; i <= 4; i++) {
  console.log("*".repeat(i));
}
```

7. Arrays, Tuples & Enums

```
let marks: number[] = [90, 80, 70];

let total = marks.reduce((sum, mark) => sum + mark, 0);

console.log(total);

let employee: [number, string, number] = [1, "Bala", 50000];

enum OrderStatus {
  Pending,
  Shipped,
  Delivered,
  Cancelled
}

console.log(OrderStatus.Delivered);
```

8. Unions, Intersections & Enums

```
let value: string | number;

value = "Playwright";
value = 100;

interface Employee {
  name: string;
}

interface Manager {
  department: string;
}

type TeamLead = Employee & Manager;
```

```
const lead: TeamLead = {
  name: "Bala",
  department: "QA"
};
```

9. Functions & Interfaces

```
function add(a: number, b: number): number {
  return a + b;
}

const multiply = (a: number, b: number): number => {
  return a * b;
};

interface Student {
  id: number;
  name: string;
  course: string;
  marks: number;
}

const student: Student = {
  id: 1,
  name: "Bala",
  course: "TypeScript",
  marks: 95
};
```

10. Mini Project - Student Management System

```
interface Student {
  name: string;
  marks: number;
}

const students: Student[] = [
  { name: "A", marks: 90 },
  { name: "B", marks: 80 }
];

students.forEach(student => {
  console.log(student.name, student.marks);
});
```

11. Bonus Challenges

```
// Palindrome Checker
function isPalindrome(word: string): boolean {
  return word === word.split("").reverse().join("");
}

console.log(isPalindrome("madam"));

// Fibonacci Series
let a = 0, b = 1;

for(let i = 0; i < 10; i++) {
  console.log(a);
  let temp = a + b;
  a = b;
  b = temp;
}
```

These sample answers demonstrate core TypeScript concepts and practical coding patterns. Learners are encouraged to enhance and optimize the solutions further.